

# PROJECT REPORT ON

## PEST CONTROL ADVISOR

INT 428

### Submitted By:

Shailja Kumari Registration Number: 12326646

Saksham Registration Number: 12305209

Gagandeep Singh Registration Number: 12322960

### Under The Guidance Of:

Mr. Sushil Lekhi

**8 April, 2025**



**L** OVELY  
**P** ROFESSIONAL  
**U** NIVERSITY

## Introduction

One of the major challenges in pest management is effective communication between users and experts. The **Pest Control Advisor Chatbot** is a web-based application designed to bridge this gap by providing real-time assistance and recommendations for pest control. Users can quickly and efficiently get advice on identifying pests, selecting appropriate treatments, and implementing preventive measures.

The chatbot offers interactive, conversational support that enhances the user experience. Additional features include user authentication, session management, and data encryption to ensure secure and personalized consultations. The goal of this project is to develop an easy-to-use and accessible digital advisor for effective pest control guidance.

**The primary objectives of the Pest Control Advisor Chatbot are:**

- Provide real-time assistance for identifying pests and suggesting control methods.
- Allow users to register and log in to access personalized pest management advice.
- The chatbot can interact with users through greetings, fun facts about pests, and useful tips.
- Maintain an uninterrupted user experience by keeping the login session active.
- Display appropriate error messages if the user's input is invalid or not recognized.

This project aims to provide a smooth and enjoyable translation experience, combining a range of functional and fun features.

## **1. Pest Advisory and Identification**

- Users input a pest-related query or description and select a category (e.g., insects, rodents, fungi)
- The system processes the request using a pest identification and recommendation database.
- Regular expressions help identify patterns and keywords in the user's input.
- If the pest or category is invalid or unsupported, the user receives an informative alert.

## **2. Chatbot-Like Interaction**

- To make the tool more engaging, chatbot-like responses are included. Greets users with friendly messages when they say "hi" or "hello."
- Responds with pest-related fun facts or tips when asked for one.
- Says goodbye with a polite message when the user exits the session.
- If input is unrecognized, the chatbot provides helpful guidance on what to do next.

## **3. Secure Session Management**

- Users stay logged in during the entire session.
- Flask session management allow users to stay logged in.

## **5. Error Handling and Validation**

- If a pest query fails to return a result, the chatbot notifies the user of the issue.
- If an unsupported pest category or invalid input is provided, an appropriate error message is shown.

## **Benefits of the Pest Control Advisor Chatbot**

1. 24/7 Availability
  - Get pest-related guidance anytime, without waiting for human assistance.
2. Instant Pest Identification & Solutions
  - Quickly identify pests and receive tailored treatment recommendations in real-time.
3. Interactive and User-Friendly
  - Chatbot-style interaction makes it engaging and easy to use, even for non-experts.
4. Error Handling and Smart Suggestions
  - Guides users even if inputs are incorrect or unclear, ensuring smooth interactions.
5. Custom Advice Based on User Input
  - Gives personalized recommendations based on the type of pest and environment described.

# FRONTEND CODE

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Pest Control Advisor Chatbot - Shailja Kumari</title>
7   <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
8   <style>
9     :root {
10       --primary-color: #4a7c59; /* Vibrant forest green */
11       --secondary-color: #8ab17d; /* Lively lime green */
12       --accent-color: #f4a261; /* Warm coral orange */
13       --light-gray: #fefae0; /* Creamy off-white */
14       --dark-gray: #2d2d2a; /* Deep charcoal */
15       --text-color: #1f271b; /* Rich dark green-gray */
16       --success-color: #76c893; /* Bright success green */
17       --box-shadow: 0 6px 15px rgba(0, 0, 0, 0.15);
18     }
19
20     body {
21       font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
22       background: linear-gradient(135deg, var(--light-gray), #e0e6d8);
23       margin: 0;
24       padding: 0;
25       display: flex;
26       justify-content: center;
27       align-items: center;
28       height: 100vh;
29       overflow: hidden;
```

# FRONTEND CODE

```
<html lang="en">
<body>
  <script>
    function truncateToSentence(text, maxWords = 10) {
      const words = text.split(' ');
      if (words.length <= maxWords) return text;
      return words.slice(0, maxWords).join(' ') + '.';
    }

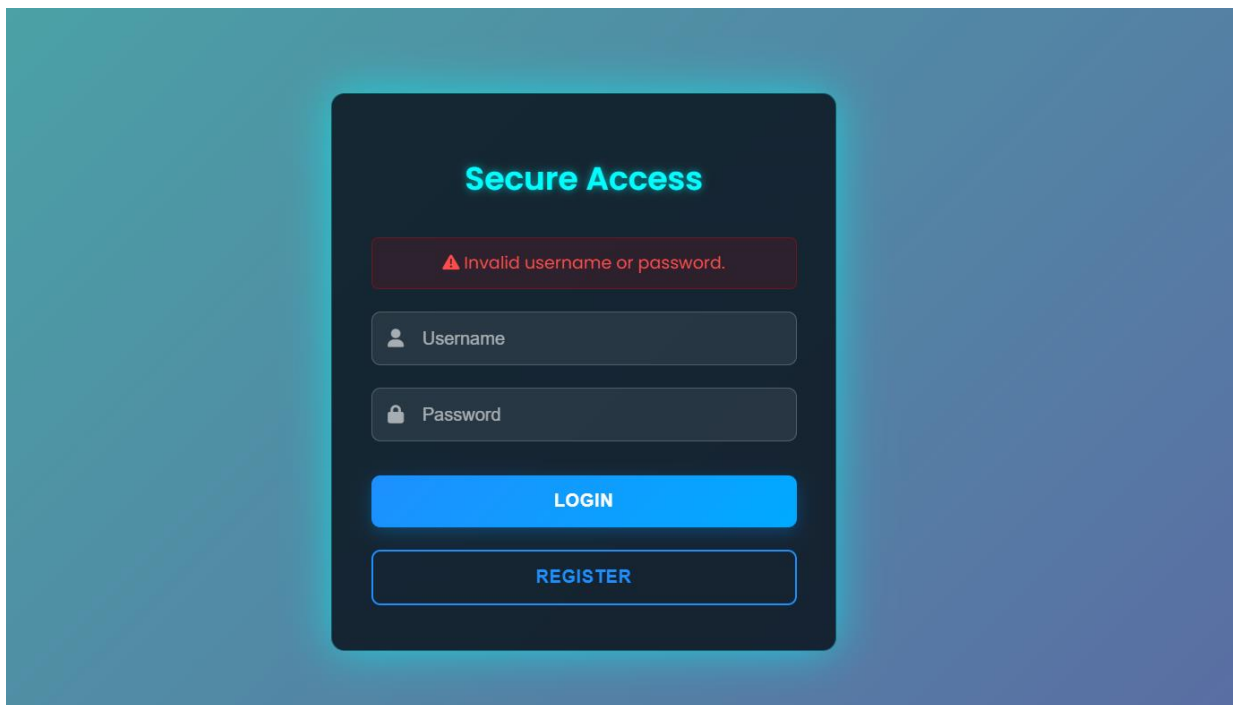
    function sendMessage() {
      const pest = pestInput.value.trim();
      if (!pest) return;

      addMessage(pest, true);
      pestInput.value = '';
      const typingIndicator = showTypingIndicator();

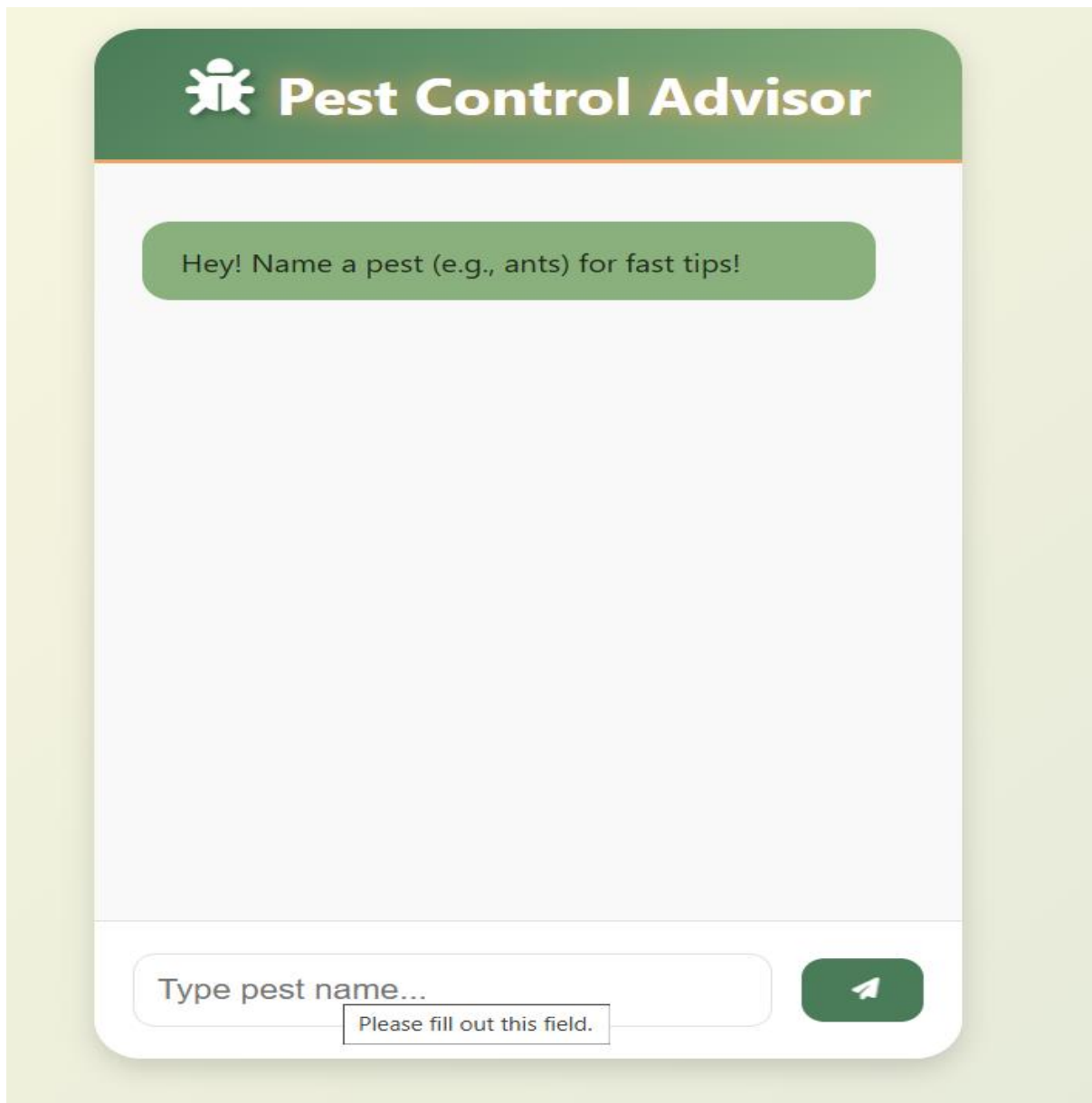
      setTimeout(() => {
        fetch('http://127.0.0.1:5000/get_advice', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ pest })
        })
        .then(response => {
          if (!response.ok) throw new Error('Network response was not ok');
          return response.json();
        })
        .then(data => {
          typingIndicator.remove();
        })
      });
    }
  </script>

```

# Screenshots of Tool

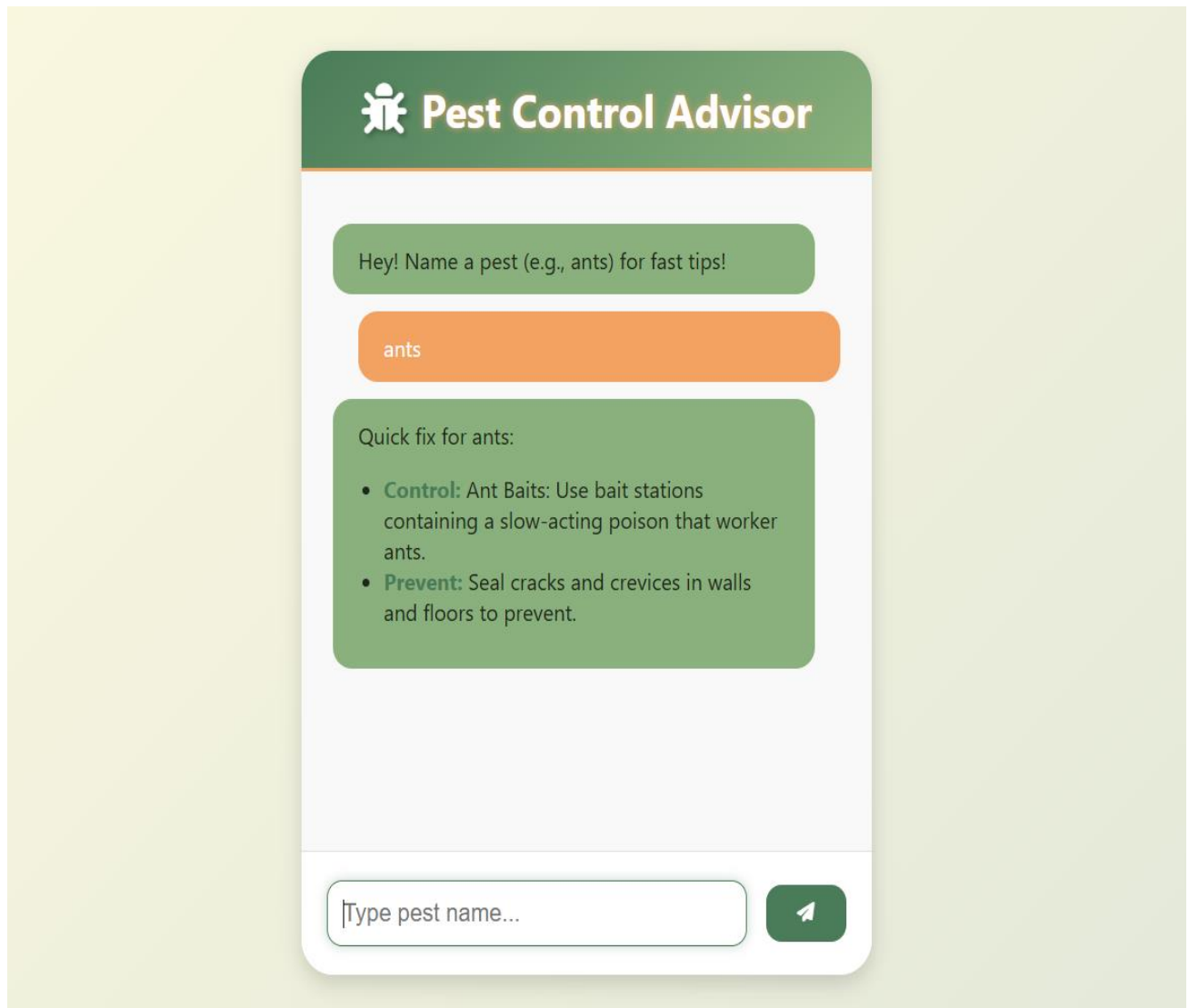


# Screenshots of Tool



The screenshot displays the 'Pest Control Advisor' tool interface. At the top, a green header bar contains a white bug icon and the title 'Pest Control Advisor'. Below this, a green rounded rectangle contains the text 'Hey! Name a pest (e.g., ants) for fast tips!'. The main area is a large, empty white space. At the bottom, there is a white input field with the placeholder text 'Type pest name...'. A small tooltip box with the text 'Please fill out this field.' is positioned over the input field. To the right of the input field is a green button with a white paper plane icon.

# Screenshots of Tool



# Screenshots of code

```
1 import os
2 import requests
3 import json
4 from flask import Flask, request, jsonify
5 from flask_cors import CORS
6
7 app = Flask(__name__)
8 CORS(app)
9
10 # Use your API Key directly
11 GEMINI_API_KEY = "AIzaSyA5mls2d8GpQBNEmtcQxcoVyBst-iUQZNo"
12
13 def get_pest_advice(pest_type):
14     api_url = f"https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent?key={GEMINI_API_KEY}"
15     headers = [{"Content-Type": "application/json"}]
16
17     # Create the prompt for pest control advice
18     prompt = f"""
19     Provide detailed pest control advice for {pest_type}. Include:
20     - identification (string): Key characteristics to identify the pest
21     - methods (array): List of 3-5 control methods with brief descriptions
22     - prevention (string): Tips to prevent future infestations
23     - safety (string): Safety precautions during treatment
24
25     Return ONLY valid JSON format like this (no other text or markdown):
26     {{
27         "advice": {{
28             "identification": "Description",
29             "methods": [
```

# Screenshots of code

```
try:
    response = requests.post(api_url, headers=headers, json=payload, timeout=15)

    if response.status_code == 200:
        response_json = response.json()

        try:
            model_response = response_json["candidates"][0]["content"]["parts"][0]["text"]

            # Extract just the JSON part
            if "```json" in model_response:
                json_start = model_response.find("{")
                json_end = model_response.rfind("}") + 1
                json_data = model_response[json_start:json_end]
            else:
                json_data = model_response.strip()

            # Parse the JSON data
            parsed_data = json.loads(json_data)

            if "advice" not in parsed_data or not isinstance(parsed_data["advice"], dict):
                return {"error": "Invalid response format from AI", "advice": {}}

            return parsed_data

        except (json.JSONDecodeError, KeyError, IndexError) as e:
```

# Screenshots of code

```
<body>

<script>
  const chatContainer = document.getElementById('chat-container');
  const userInput = document.getElementById('user-input');
  const sendButton = document.getElementById('send-button');
  const speakButton = document.getElementById('speak-button');
  const targetLanguageSelect = document.getElementById('target-language');

  // --- Speech Synthesis Setup ---
  const synth = window.speechSynthesis;
  let voices = [];

  function populateVoiceList() {
    if(typeof speechSynthesis === 'undefined') {
      console.warn("Speech Synthesis not supported by this browser.");
      speakButton.disabled = true; // Disable button if not supported
      // Style updates are now handled by CSS :disabled selector
      return;
    }
    voices = synth.getVoices();
    // console.log("Available voices:", voices); // Optional: log voices
  }

  populateVoiceList();
  // Some browsers need this event to load voices asynchronously
  if (typeof speechSynthesis !== 'undefined' && speechSynthesis.onvoiceschanged !== undefined) {
    speechSynthesis.onvoiceschanged = populateVoiceList;
  }

  // --- Event Listeners ---
  sendButton.addEventListener('click', sendMessage);

  // Send message on Enter key press (but not Shift+Enter)
  userInput.addEventListener('keydown', (event) => {
    if (event.key === 'Enter' && !event.shiftKey) {
      event.preventDefault(); // Prevent adding a new line
    }
  });

```

Ln 103, Col 36 Spaces: 4 UTF-8 CRLF Django HTML Completions limit reached Go Live

# Screenshots of code

```
except requests.exceptions.RequestException as e:
    return {"error": f"Request error: {str(e)}", "advice": {}}

@app.route('/get_advice', methods=['POST'])
def pest_advice():
    try:
        data = request.get_json()

        # Extract pest type
        pest_type = data.get("pest", "").strip()

        if not pest_type:
            return jsonify({"error": "Pest type is required", "advice": {}})

        # Get pest control advice
        advice = get_pest_advice(pest_type)

        return jsonify(advice)

    except Exception as e:
        return jsonify({"error": f"Server error: {str(e)}", "advice": {}})

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
    app.run(debug=True, host='0.0.0.0', port=port)
```

## Conclusion

The Pest Control Advisor Chatbot is a practical and interactive web application designed to create an easy and effective communication channel between users and pest control support. Through real-time advisory responses and chatbot interactions, the tool enhances the user experience while ensuring secure access with robust authentication techniques. Future enhancements could include

- Image input or input through a web cam
- **Offline Mode:** Pest control advisor model without internet access.
- **AI-Powered Real Time Video input:** For better user interaction.

The Pest control advisor is a AI tool that helps users tackle Pest-related issues offering expert advice and making pest management simpler and more accessible for everyone.

## References

1. Gemini API Documentation: For fetching data and providing services
2. Flask Framework: For web development and session management
3. Regular Expressions (Python re Module): For input validation

### **GITHUB LINK OF THE CHATBOT :**

<https://github.com/Shailja12326646/pest-control-advisor>

<https://github.com/Rio-07/AI-chatbot-for-pest-control-advisory>